

# Medicine Counting Machine Operations Handbook

System overview, operator workflows, architecture, and deployment guidance  
for the Pill Count UI platform



|               |                                                     |
|---------------|-----------------------------------------------------|
| Source        | Project README provided by the user                 |
| Document type | Book-style technical handbook                       |
| Audience      | Operators, supervisors, engineers, and implementers |

## What this handbook covers

This edition turns the original README into a structured reference manual. It explains the product purpose, tab-by-tab behavior, backend composition, live event model, data design, operational flows, local setup, and deployment notes in a format that is easier to browse, print, and share.

# Contents

This handbook is organized for readers who need both business context and technical implementation detail. The early chapters focus on product behavior, while the later chapters cover engineering and deployment.

1. Executive overview
2. Product purpose and operating model
3. Dashboard tabs and operator experience
4. System architecture and live updates
5. Backend modules and data model
6. Inventory, jobs, and machine lifecycle
7. Runtime flows
8. Repository structure and active code paths
9. Local development guide
10. Environment, deployment, and appendix

## Reading path

Operators can start with Chapters 2, 3, and 7. Engineers onboarding to the codebase can jump from Chapters 4 through 10. Administrators and compliance reviewers will usually care most about the audit, RBAC, API key, and deployment sections.

## 1. Executive Overview

The platform is an operations console for medicine and pill counting. It combines a web dashboard for staff, a NestJS API for domain logic, PostgreSQL for persistence, and a live event stream so the dashboard can reflect machine and inventory activity without relying purely on manual refreshes.

At the business level, the system supports authentication, machine registration, counting jobs, stock control by pill and lot, FEFO-oriented dispensing, and an append-only audit trail for write activity. At the implementation level, it is a monorepo with a Next.js frontend, a NestJS backend, shared schemas, and reusable UI primitives.

## Key strengths

The design balances operational speed with traceability. Inventory updates are written to a ledger and projected into current balances, live events inform dashboards in near real time, and role-based access control isolates dangerous capabilities such as the low-level Admin editor.

## 2. Product Purpose and Operating Model

The system has two primary actors: human operators using the web console and machine clients sending registration, heartbeat, and counting events into the API. This split explains much of the architecture. The dashboard exists for supervised operational control, while the machine endpoints

exist for reliable ingestion, status tracking, and event logging.

Core operational responsibilities

- Authenticate pharmacy or warehouse staff and expose features according to assigned roles.
- Register counting machines, monitor heartbeat health, and inspect event payloads.
- Create counting jobs, start them, record progress, complete them, or trigger recounts when tolerance is exceeded.
- Track stock by pill type, lot, expiry, location, and movement type.
- Apply FEFO-style allocation so eligible lots with the earliest expiry are dispensed first.
- Maintain auditability for successful write operations across the platform.

Operational status

| Area                                                                                          | Status                     | What it means                                                                                |
|-----------------------------------------------------------------------------------------------|----------------------------|----------------------------------------------------------------------------------------------|
| Overview, Jobs, Inventory, Lots, Machines, Reports, Users & Roles, Admin, Settings, Audit Log | Live / implemented         | These tabs have active behavior in the current application.                                  |
| Maintenance                                                                                   | Present but disabled       | Reserved for future maintenance scheduling and service workflows.                            |
| Notifications                                                                                 | Present but disabled       | Reserved for future alert routing and notification management.                               |
| Webhook rules and tolerance profile panels                                                    | Informational placeholders | Visible in Settings as future-phase concepts rather than completed tooling.                  |
| API automated tests                                                                           | Placeholder                | The build and lint scripts exist, but the API test script is not yet a real automated suite. |

3. Dashboard Tabs and Operator Experience

The dashboard shell is the best source of truth for the active navigation. In practice, the tabs are arranged to mirror how operations teams work: observe the overall state, run jobs, maintain stock accuracy, inspect lot health, supervise machines, and then manage roles, settings, and audit records.

| Tab             | Purpose                        | Current behavior                                                                                                            |
|-----------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Overview        | Executive snapshot             | KPI cards, latest jobs, recent machine events, fleet snapshot, CSV export, and manual refresh.                              |
| Jobs / Sessions | Counting work management       | Create jobs, assign machines and pill types, start jobs, complete jobs, and trigger recount workflows when tolerance fails. |
| Inventory       | Stock ledger operations        | Receive, transfer, adjust, reserve, release, and dispense stock while exposing balances and movement history.               |
| Lots & Expiry   | Lot control and FEFO readiness | Create lots, expose expiry risk, quarantine or release lots, and view FEFO priority order by soonest expiry.                |

| Tab            | Purpose                     | Current behavior                                                                                                               |
|----------------|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Machines Fleet | Machine supervision         | Shows registration state, firmware, last heartbeat, computed online/offline visibility, and a searchable event inspector.      |
| Reports        | Analytics and exports       | Aggregates throughput, valuation by location, and exportable job and inventory data.                                           |
| Users & Roles  | RBAC administration         | Displays user directory, role assignments, and permission matrix; role assignment currently uses a direct role ID prompt flow. |
| Admin          | Low-level database editor   | Admin-only direct row editor generated from Prisma metadata; bypasses normal business workflows.                               |
| Settings       | Operational configuration   | The live capability today is API key management for machine and API clients.                                                   |
| Audit Log      | Traceability and compliance | Append-only audit record view with actor type, action, resource, and IP metadata.                                              |

### Authentication screens

Outside the dashboard shell, the product includes Login and Register screens. Both rely on shared Zod schemas from the shared package, and registration signs the new user in immediately after account creation.

## 4. System Architecture and Live Updates

The architecture is intentionally straightforward. The browser UI talks to the API through a client that can use a same-origin proxy or a direct API URL. The NestJS API applies validation and business rules, persists state through Prisma into PostgreSQL, and publishes in-process live events for dashboards that subscribe through Server-Sent Events.

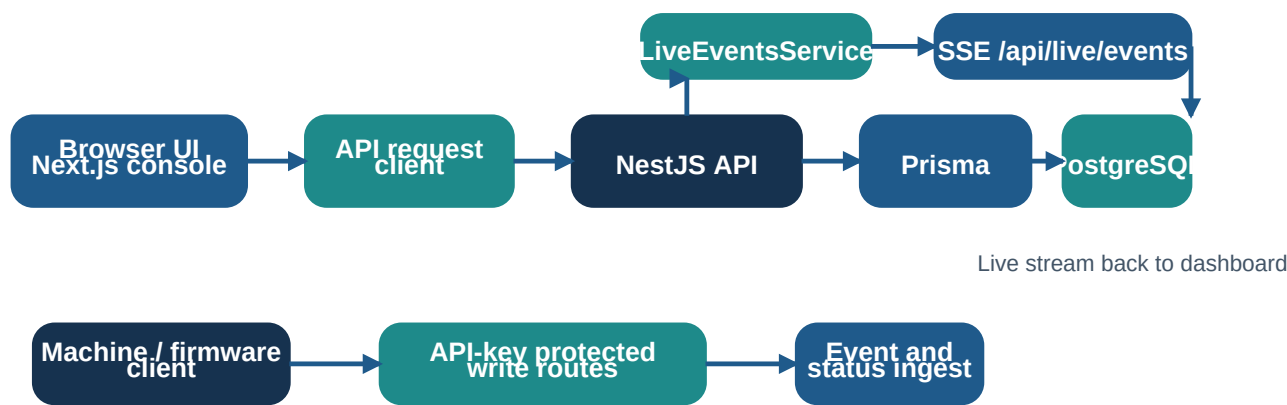


Figure 1. Simplified application and device event flow.

### API target strategy

The web client tries several API targets in sequence: the previously successful API base URL, same-origin /api for proxied deployments, NEXT\_PUBLIC\_API\_URL when configured, and http://localhost:4000/api during local development. This design makes the same frontend viable for local use, proxied deployments, and demo mode.

#### Live updates are SSE-based

Realtime behavior uses Server-Sent Events rather than WebSockets. The live controller exposes GET /api/live/events, the frontend opens a single EventSource, and the dashboard surfaces stream health via a connection badge and banner. This keeps the live model operationally simple while still supporting status, inventory, and job notifications.

### Representative live event types

| Category                     | Examples                                                                                                             |
|------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Machine health and telemetry | machine.status.changed, machine.event, machine.error                                                                 |
| Inventory operations         | inventory.received, inventory.transferred, inventory.dispensed, inventory.low_stock, inventory.cycle_count.completed |
| Job workflow                 | job.created, job.started, job.progress, job.completed, job.recount.required                                          |
| Expiry monitoring            | expiry.alert                                                                                                         |

## 5. Backend Modules and Data Model

The API is organized by domain modules rather than by a single broad controller layer. This keeps business concerns isolated and makes it easier for maintainers to navigate the codebase. The main functional modules are auth, users, rbac, machines, events, pill-types, lots, inventory, jobs, reports, audit, api-keys, live, and admin.

Database model groups

| Model group         | Representative entities                               | Why it exists                                                                             |
|---------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Identity and access | User, Role, UserRole, RefreshToken, ApiKey            | Supports authentication, token refresh, roles, and machine/API credentials.               |
| Machine operations  | Machine, MachineEvent                                 | Stores machine registration state, heartbeats, and ingested event payloads.               |
| Product and stock   | PillType, Lot, InventoryBalance, InventoryTransaction | Separates descriptive product data, lot governance, current balances, and ledger history. |
| Counting execution  | CountingJob, JobProgress, JobEvidence                 | Tracks the lifecycle and evidence trail of counting work.                                 |
| Stock verification  | CycleCountSession, CycleCountItem                     | Supports cycle count and reconciliation flows.                                            |
| Compliance          | AuditLog                                              | Preserves a durable trace of successful write operations.                                 |

Why the inventory model matters

Inventory is split between an append-only ledger and a current-state projection. This allows the interface to read balances quickly while preserving the full historical chain needed for audits, reporting, and reconciliation.

## 6. Inventory, Jobs, and Machine Lifecycle

### Inventory rules

Inventory write paths support receive, transfer, adjust, reserve, release, dispense, and cycle count. These operations run inside database transactions and use idempotency keys so that retries do not accidentally duplicate business actions. Quarantined lots are excluded from dispense allocation, reserved and quarantined quantities reduce available stock, and low-stock events can be emitted when balances cross the configured threshold.

Dispense logic is FEFO-aware. Eligible lots are sorted by earliest expiry date, allowing the platform to dispense the stock most in need of rotation first. This is a critical behavior for environments where expiry exposure matters as much as count accuracy.

### Job lifecycle

Counting jobs are created with a machine, pill type, target quantity, tolerance, and an optional preferred lot. A job can be started, updated with progress, and completed with an actual count. The resulting variance is checked against tolerance. Jobs within tolerance are marked completed and emit a `job.completed` event. Jobs outside tolerance are marked `NEEDS_RECOUNT` and can generate linked recount work.

### Machine lifecycle

Machine clients can register, send heartbeat updates, and ingest event payloads. Registration upserts the machine and marks it online. Heartbeats refresh `lastSeen`, maintain online status, and log a heartbeat event. Displayed online/offline state is computed from the last-seen time and the offline threshold configuration, which is why the fleet page can show practical status rather than only the last stored state flag.

| Lifecycle             | Inputs                                                             | Outputs / effects                                                                                |
|-----------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Inventory transaction | Receive, transfer, adjust, reserve, release, dispense, cycle count | Ledger write, balance update, business validation, optional live event, optional low-stock alert |
| Counting job          | Target qty, tolerance, machine, pill type, actual count            | Status transitions, recount requirement, live completion event when in tolerance                 |
| Machine event         | Register, heartbeat, telemetry payload, idempotency key            | Machine upsert, event persistence, deduplication, online/offline visibility update               |

## 7. Runtime Flows

### Human operator flow

- A user signs in through the Login page and the frontend stores access and refresh tokens in browser local storage.
- The dashboard loads queries through React Query and renders the authenticated console shell.
- The user performs an action such as job creation or stock dispense.
- The request is validated and executed by the API, persisted to PostgreSQL, and reflected in audit and live event systems.

- The UI updates through a combination of refetches and SSE-driven awareness.

## Machine flow

- A device authenticates using an API key on machine write routes.
- The device registers itself or sends heartbeat updates to maintain active status.
- Machine event payloads are written to persistent storage with idempotency-based deduplication.
- The API publishes resulting live updates so connected dashboards can observe fleet activity immediately.
- Operators inspect the event history from the Machines Fleet screen as needed.

### Audit perspective

Every successful write request except the SSE endpoint is observed by audit middleware. Records include actor type, actor identifier, action, resource type, resource ID, request ID, IP address, user agent, and response metadata. This makes the Audit Log tab the compliance view of almost every meaningful state change.

## 8. Repository Structure and Active Code Paths

The active runtime is the combination of `apps/web` and `apps/api`. The repository also contains legacy and older prototype code, but the README explicitly warns that those areas are not the primary production path. For maintainers, the fastest onboarding route is to start with navigation and page structure in the web app, then inspect API composition and the Prisma schema.

|                                           |                                               |
|-------------------------------------------|-----------------------------------------------|
| <code>apps/</code>                        |                                               |
| <code>  api/</code>                       | NestJS API + Prisma + PostgreSQL integration  |
| <code>  web/</code>                       | Next.js App Router web console                |
| <code>packages/</code>                    |                                               |
| <code>  shared/</code>                    | shared zod schemas, constants, and types      |
| <code>  ui/</code>                        | reusable UI building blocks                   |
| <code>data/</code>                        |                                               |
| <code>  live-edit.sql</code>              | watched SQL scratchpad for local/live DB work |
| <code>  live-current.md</code>            | exported database snapshot notes              |
| <code>docs/</code>                        |                                               |
| <code>  sample-event-payloads.json</code> |                                               |
| <code>legacy/</code>                      | older code retained for reference             |

### Recommended onboarding order

1) `apps/web/components/app-shell.tsx` for navigation and tabs. 2) `apps/web/app/(dashboard)` for per-page behavior. 3) `apps/web/lib/api.ts` for request flow. 4) `apps/api/src/app.module.ts` for backend composition. 5) `apps/api/prisma/schema.prisma` for the domain model.



## 9. Local Development Guide

### Prerequisites

- Node.js and npm installed locally.
- A local PostgreSQL instance.
- PowerShell if the Windows helper scripts will be used.

### Install and environment setup

```
npm install

# Copy environment templates
Copy-Item apps\api\.env.example apps\api\.env -Force
Copy-Item apps\web\.env.example apps\web\.env.local -Force
```

### Database preparation

```
.\scripts\setup-local-db.ps1

# or create manually
CREATE ROLE "pillcount" LOGIN PASSWORD 'pillcount';
CREATE DATABASE "pillcount" OWNER "pillcount";
GRANT ALL PRIVILEGES ON DATABASE "pillcount" TO "pillcount";
```

### Generate client, push schema, seed data, and run

```
npm run prisma:generate
npm run prisma:push
npm run prisma:seed
npm run dev
```

| Default local endpoint | Value                               |
|------------------------|-------------------------------------|
| Web console            | http://localhost:3100               |
| API base               | http://localhost:4000/api           |
| One-command startup    | npm run dev:local                   |
| Separate startup       | npm run dev:api and npm run dev:web |

#### Live SQL watcher

The root development flow also runs `db:watch`, which watches `data/live-edit.sql` and applies changes through a PowerShell helper. This is a local operator convenience for database work and not part of the main request path of the application.

## 10. Environment, Deployment, and Appendix

### Important environment variables

| Layer | Representative variables                                                                                                                                   | Role                                                                                                       |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| API   | DATABASE_URL, JWT_ACCESS_SECRET, JWT_REFRESH_SECRET, JWT_ACCESS_TTL, JWT_REFRESH_TTL, LOW_STOCK_THRESHOLD, EXPIRY_ALERT_DAYS, MACHINE_OFFLINE_THRESHOLD_MS | Database connectivity, token security, stock thresholding, expiry alerting, and machine liveness behavior. |
| Web   | API_PROXY_TARGET, NEXT_PUBLIC_API_URL, NEXT_PUBLIC_STORE_NAME, NEXT_PUBLIC_STORE_LOGO_URL, NEXT_PUBLIC_DEMO_MODE                                           | API routing, branding, and demo-mode behavior.                                                             |

Deployment notes

The project can be launched with Docker Compose for local or containerized setups. For Vercel, the API and web app are expected to deploy as separate projects from the same repository. The API still needs an external PostgreSQL database, and the web app will typically point to the API through `API_PROXY_TARGET=https://<api-project>/api`.

```
docker compose up --build
```

Seeded access and references

| Reference        | Value                                                                                                                                                     |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Seed admin email | admin@pillcount.local                                                                                                                                     |
| Seed password    | Admin1234!                                                                                                                                                |
| Seed machine key | mch_live_seed_key_123456789                                                                                                                               |
| Useful files     | docs/sample-event-payloads.json; apps/api/prisma/schema.prisma; apps/web/lib/api.ts; apps/web/hooks/use-live-events.ts; apps/web/components/app-shell.tsx |

Final takeaway

At its core, this medicine counting machine platform is both an operational dashboard and a domain API. The frontend structures human workflows, the backend enforces business rules, PostgreSQL preserves both current and historical state, and SSE keeps the dashboard aware of live machine and inventory changes.

End of handbook